

# Architecture Proposal: Automated WPL 02 Form Processing Pipeline

**Objective:** To adapt the existing crazyOCR batch-processing pipeline to autonomously extract structured text data (student details, ELO scores) and image snippets (signatures, dates) from physically scanned WPL 02 Mentor Assessment forms. The final output will be a structured CSV dataset and categorised image directories, ready for downstream PDF template generation.

## Part 1: Executive Overview (Layman's Terms)

Our current text-extraction system (crazyOCR) is designed to read continuous blocks of text, much like reading a book. However, processing structured forms like the WPL 02 Mentor Assessment requires a different approach. Because these forms are printed, signed by hand, and scanned back in, the system will encounter crooked pages, off-center scans, and varying resolutions.

If we simply tell the system to "read the page," it will jumble the table columns together, losing the critical relationship between a specific ELO (Exit Level Outcome) and its corresponding Q1/Q2 score.

To solve this, we are upgrading the system from a "Whole-Page Reader" to a "Precision Stencil" approach. Here is how the new process will work:

1. **Digital Straightening (Alignment):** When a scanned form enters the system, it will likely be slightly rotated or shifted. The system will look for visual anchors (like the CPUT logo and the main table borders) to automatically rotate and stretch the scan so it perfectly matches a pristine, digital "Master Template."
2. **Applying the Stencil (Targeted Zones):** Once the page is perfectly locked into position, the system applies a digital stencil. Instead of reading the whole page, it will look through specific "cutouts" in the stencil to find exact data points: one box for the Student Name, one box for ELO 1 (Q1), etc.
3. **Smart Reading & Sorting:**
  - \* **Text Data:** The system will read the text inside the boxes. For the score columns, it will be strictly limited to looking for numbers (0-9), preventing it from mistaking a "5" for an "S".
  - o **Image Data:** Instead of reading the mentor's signature and the handwritten date, the system will take a "digital screenshot" of those specific boxes.
4. **Filing and Exporting:** The system uses the extracted Student Number to save the signature and date images neatly into folders (e.g., 123456789.png). Finally, it compiles all the text scores for the entire batch of documents into a single, clean CSV spreadsheet.

This spreadsheet and the organised image folders will then seamlessly feed into your final PDF generation script.

## Part 2: Technical Specification

To achieve this workflow, we will refactor the crazyOCR architecture, specifically modifying the pre-processing and worker execution threads to utilise **Template Matching** and **Region of Interest (ROI) processing**.

### 1. Geometric Registration (Overhauling detect\_page\_and\_crop)

Contour-based cropping is unreliable for scanned physical forms where paper edges might be cut off by the scanner. We will implement Feature-Based Alignment to standardize the geometry of every input matrix.

- **Reference Initialization:** We will define a master\_template.jpg (a blank, perfectly aligned WPL 02 form).
- **Feature Extraction:** Utilize OpenCV's ORB (Oriented FAST and Rotated BRIEF) or SIFT to detect keypoints and compute descriptors on both the master template and the input scan.
- **Homography Transformation:** \* Match features using cv2.BFMatcher or FLANN.
  - Calculate the transformation matrix using cv2.findHomography(..., cv2.RANSAC).
  - Warp the input scan using cv2.warpPerspective to exactly match the (width, height) of the master template. This guarantees pixel-perfect alignment for downstream cropping.

### 2. ROI Coordinate Mapping

We will define a configuration dictionary containing the bounding box coordinates (x, y, w, h) mapped to the Master Template's dimensions.

```
ROI_MAP = {  
    "student_number": {"x": X, "y": Y, "w": W, "h": H, "type": "text", "numeric_only": True},  
    "student_name": {"x": X, "y": Y, "w": W, "h": H, "type": "text", "numeric_only": False},  
    "elo1_q1": {"x": X, "y": Y, "w": W, "h": H, "type": "text", "numeric_only": True},  
    # ... all ELOs and Totals ...  
    "mentor_signature": {"x": X, "y": Y, "w": W, "h": H, "type": "image", "dir": "signatures"},  
    "handwritten_date": {"x": X, "y": Y, "w": W, "h": H, "type": "image", "dir": "handwritten_date"}  
}
```

### 3. Execution Pipeline Refactor (ocr\_worker)

The ocr\_worker thread will transition from a global Pytesseract call to a targeted loop iterating over the ROI\_MAP.

- **Execution Order:** The worker must process and validate the student\_number first, as this string acts as the primary key for the image exports.
- **Tesseract Reconfiguration:**
  - When passing an ROI snippet to Tesseract, the Page Segmentation Mode must be updated to --psm 7 (Treat image as a single text line) or --psm 8 (Treat image as a single word).
  - For numeric fields (Student Number, ELOs, Totals), we will inject -c tesseract\_char\_whitelist=0123456789 into the OCR\_CONFIG to drastically reduce false-positive character recognition.
- **Image Snapping:** When the loop encounters a "type": "image" ROI, it will bypass OCR entirely. Instead, it will use cv2.imwrite() to dump the cropped tensor to disk using the schema: {dir}/{student\_number}.png.

### 4. Data Aggregation & Output

The multi-threaded batch\_process\_book function will be updated to aggregate Python dictionaries rather than raw strings.

- **Validation Guardrails:** Apply a Regex or length check on the extracted Student Number. If validation fails, the script will skip image generation for that document and append the filepath to an error\_log.txt to prevent corrupted outputs.
- **CSV Compilation:** Upon ThreadPoolExecutor completion, the compiled list of dictionaries will be passed to pandas.DataFrame.to\_csv() or csv.DictWriter, generating the final flat-file database formatted explicitly for the downstream PDF widget templating system.